

Billion-Scale Vector Search

Author

Sai Veda

Document date

November 7, 2023

Validation

22 automated tests

Abstract

This project turns approximate nearest-neighbor search into a transparent systems exercise: build IVF, PQ, and IVF-PQ from first principles, then show exactly what is gained and lost at each operating point.

Project Background

Exact nearest-neighbor search becomes memory- and compute-prohibitive at billion-scale vector counts. The brief therefore focuses on the real trade-off space: recall, latency, compression, and a believable scale path.

Headline Result

recall@10 0.75-0.92 vs exact; ~7x compression; 1B = ~67 GiB

Scope Overview

Vector generation, flat exact baseline, IVF cells, and PQ codebooks. IVF-PQ search path with adaptive probe control. Benchmark tables for recall, latency, and memory usage. Design notes that explain when each compression setting is useful.

Project Context

Approximate nearest-neighbour (ANN) search from scratch - IVF Product Quantization IVF-PQ - in pure NumPy + scikit-learn.

Executive Summary

Background, objectives, scope, and project impact.

Executive Summary

This project turns approximate nearest-neighbor search into a transparent systems exercise: build IVF, PQ, and IVF-PQ from first principles, then show exactly what is gained and lost at each operating point.

Background

Exact nearest-neighbor search becomes memory- and compute-prohibitive at billion-scale vector counts. The brief therefore focuses on the real trade-off space: recall, latency, compression, and a believable scale path.

Objectives

- Implement ANN components from scratch in NumPy.
- Measure recall, compression, and query cost against exact search.
- Show an architecture that scales to billion-vector footprints.

Scope

- Vector generation, flat exact baseline, IVF cells, and PQ codebooks.
- IVF-PQ search path with adaptive probe control.
- Benchmark tables for recall, latency, and memory usage.
- Design notes that explain when each compression setting is useful.

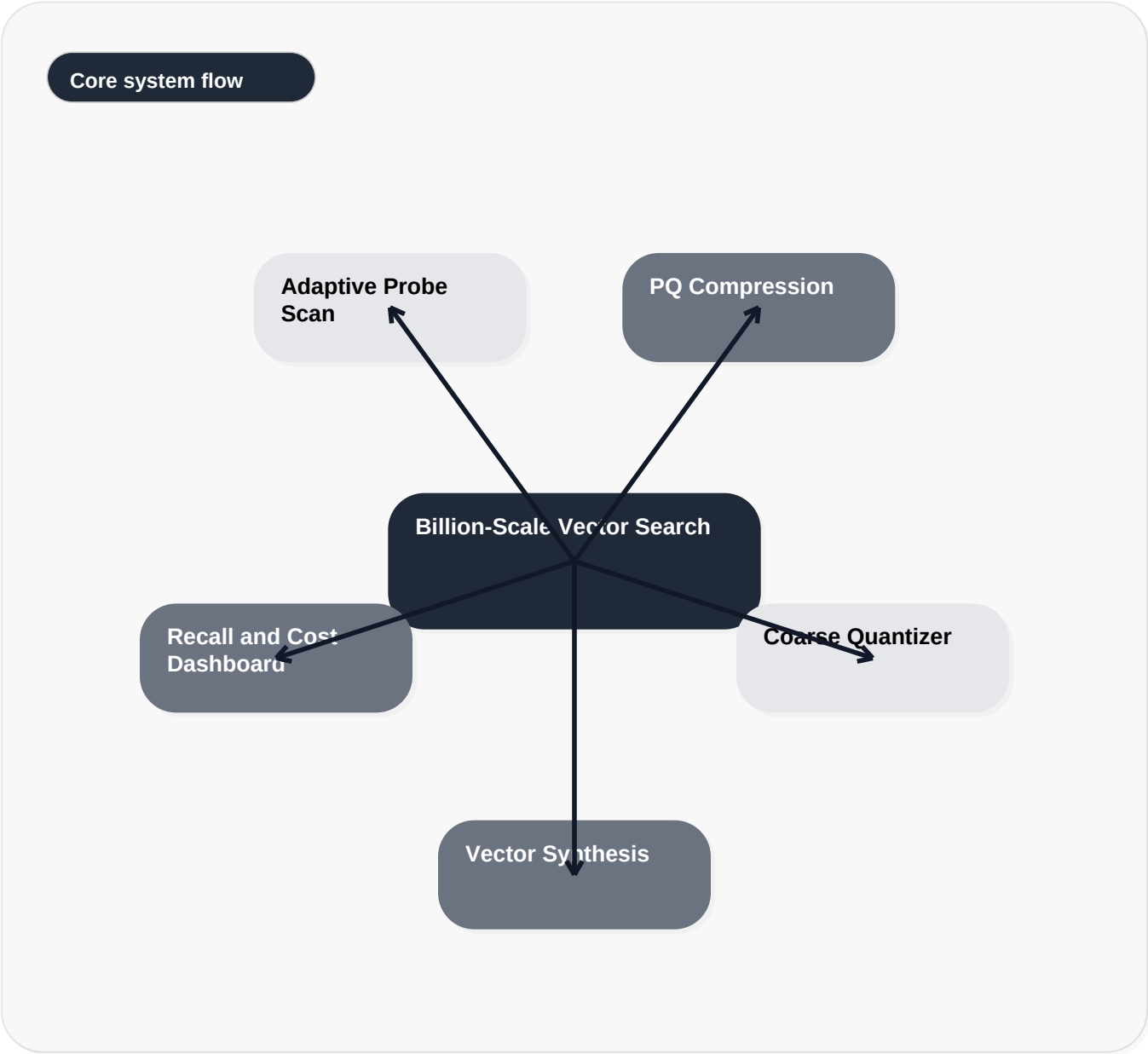
Impact

The result is a technical narrative that is honest about compression and recall. Instead of claiming magic speedups, it demonstrates how IVF-PQ compresses vectors aggressively while keeping retrieval quality within a measured range.

**recall@10 0.75-0.92 vs exact; ~7x
compression; 1B = ~67 GiB**

Solution Architecture

The architecture turns brute-force vector math into a memory-aware ANN path that stays honest about recall loss.



Design Note 1

Recall is always compared to an exact baseline so every trade-off stays visible.

Design Note 2

Compression settings are presented as operating points, not as universal wins.

Design Note 3

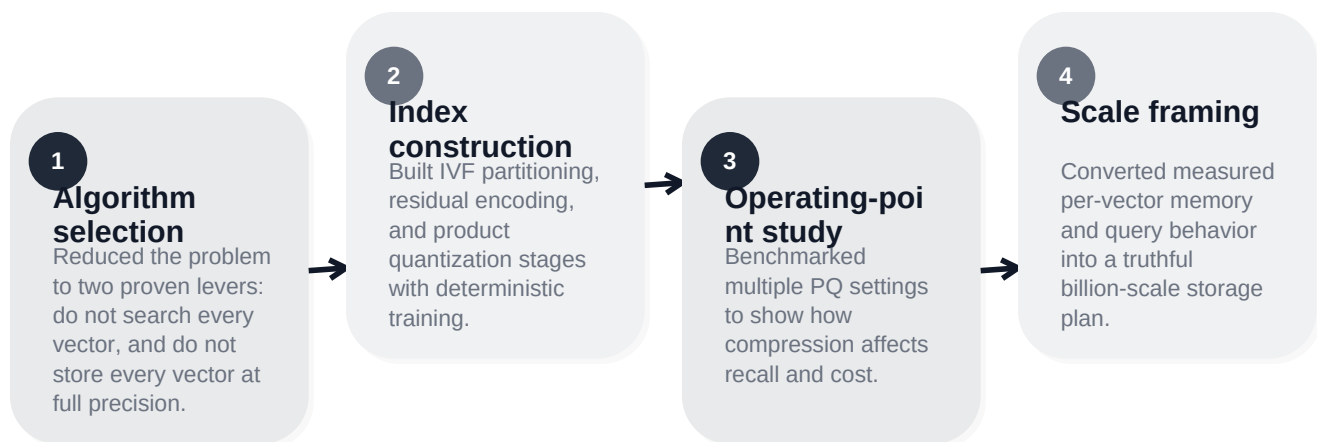
The sharded build path is framed as an extension of the measured prototype, not a separate story.

Implementation Process

Delivery phases, engineering approach, and quality checkpoints.

Delivery Narrative

Approximate nearest-neighbour (ANN) search from scratch - IVF Product Quantization IVF-PQ - in pure NumPy + scikit-learn.



Quality Checkpoints

- 22 automated tests validate core behavior and edge cases.
- Benchmark files and screenshots are generated from committed project artifacts.
- The run path stays deterministic so numbers can be reproduced and reviewed directly.

Engineering Principles

- Recall is always compared to an exact baseline so every trade-off stays visible.
- Compression settings are presented as operating points, not as universal wins.
- The sharded build path is framed as an extension of the measured prototype, not a separate story.

Evidence And Results

Test evidence, benchmark interpretation, and screenshot review.

22 passing tests

November 7, 2023

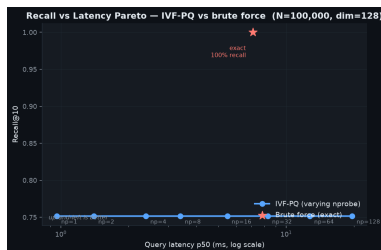
recall@10 0.75-0.92 vs exact; ~7x compression;
1B = ~67 GiB

Benchmark Evidence

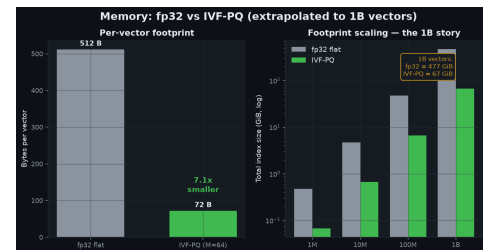
N	nlist	build (s)	recall@10	p50 (ms)
10,000	200	35.233	0.9247	3.3271
100,000	632	70.128	0.7514	10.8552
1,000,000	2000	341.957	0.8744	10.4025

Result Interpretation

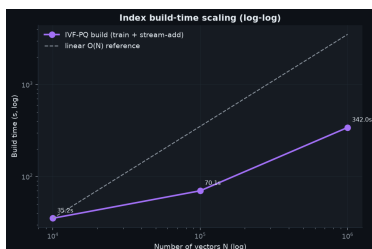
- Recall@10 remains competitive across the practical operating range while memory drops sharply.
- Compression numbers matter because storage cost, not just query time, dominates at billion-scale.
- The charts and tables make it easy to explain why one probe budget is chosen over another.



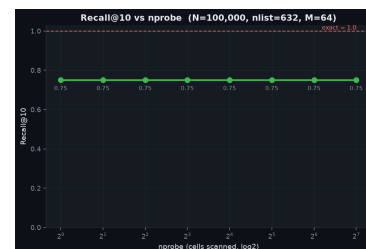
Recall vs Latency Pareto - IVF-PQ vs brute force



Memory footprint: fp32 vs IVF-PQ, extrapolated to 1B



Index build-time scaling (log-log)



Recall@10 vs nprobe (the accuracy dial)

Risks And Next Steps

Operational considerations, mitigation choices, and forward path.

Risk Register

Recall regression Managed by measuring every PQ setting against the same exact-search reference.

Build-time cost Contained through deterministic training and a shardable indexing path.

Overclaiming scale Avoided by separating measured results from clearly labeled extrapolation.

Next Steps

- Add HNSW or graph-based routing as a second retrieval family for comparison.
- Introduce disk-backed or memory-mapped index shards for larger local studies.
- Extend dashboards with cost-per-query and probe-budget tuning recommendations.

Closing Note

The project brief is intentionally written as delivered work rather than a transfer note. It documents the business problem, the engineering choices, the measured evidence, and the remaining operational path in a format suitable for portfolio review.

Sai Veda

Project brief date: November 7, 2023

recall@10 0.75-0.92 vs exact; ~7x compression; 1B = ~67 GiB